

UNIVERSITY OF IOANNINA

Department of Computer Science and Engineering

Development of an Application for Supporting Online Purchase Decisions

Master's Thesis

Gkovaris Christos-Grigorios

Supervisor

Anastasiadis Stergios

Ioannina, October 2026

Summary

Online store wish lists allow users to save products, but they typically do not help them decide which purchase should take priority, which option fits within their available budget, or which combination of products is realistic. The problem becomes more complex when a list contains many heterogeneous products, because the user must simultaneously consider price, need, discount, availability, and personal priorities. Research on choice overload shows that difficulty does not depend solely on the number of options, but also on the complexity of the set, the uncertainty of preferences, and the difficulty of the decision-making task [1].

In this thesis, we designed and implemented a Chrome browser extension (**Chrome Extension**) that operates alongside the **Skroutz** favorites list. The application imports products from the open favorites page, cleans and normalizes the data, and stores it locally so that deterministic decision-support rules can be applied. The user specifies a base budget, a budget flexibility percentage (**Budget Flexibility**), an urgency level, and weights for urgency and discount. The scoring mechanism calculates a priority score (**Priority Score**), budget compatibility, relative value for money, and the total cost of the proposed purchases. In parallel, the application generates purchase priorities that do not exceed the active budget and limits the selection to one product per detected category.

The implementation avoids artificial intelligence and machine learning. This choice is not presented as a general advantage, but as a deliberate limitation of the specific project: the rules are verifiable, the same data produce the same result, and the user can see the factors that influenced the ranking. For the evaluation, an automated suite of 45 tests was used, organized into five test files and executed using Python unittest and Node.js against the actual JavaScript logic. All 45 tests were completed successfully. The results demonstrate the consistency of the implementation in the tested scenarios, without constituting evidence that the system generally improves the quality of purchasing decisions, as no experiment involving end users was conducted.

Keywords: *online purchase decision support, wish list, Chrome extension, multi-criteria scoring, budgeting, explainability, deterministic rules.*

Abbreviations and Terms

Abbreviations / Term	Definition
DOM	Document Object Model
API	Application Programming Interface
UI	User Interface
MV3	Manifest V3
VFM	Value for Money
WLC	Weighted Linear Combination
MCDM	Multi-Criteria Decision Making

Πίνακας 0.1: Abbreviations and terms

Table of Contents

Chapter 1. Introduction 7

1.1 Motivation and Problem Definition 7

1.2 Objective of the Thesis 7

1.3 Main Contributions 7

1.4 Scope and Assumptions 8

1.5 Structure of the Thesis 9

Chapter 2. Theoretical Background and Related Work 10

2.1 Decision Support in Large Sets of Alternatives 10

2.2 Recommender Systems, Algorithmic Recommendations, and Transparency 11

2.3 Optimization Under Budget Constraints 12

2.4 Chrome Extensions and DOM Access 13

2.5 Positioning of the Proposed Solution 14

Chapter 3. Requirements Analysis and Design 16

3.1 Functional Requirements 16

3.2 Non Functional Requirements 17

3.3 From the Initial Concept to the Final Version 18

3.4 System Architecture 20

3.5 Product Data Model 21

3.6 Privacy and Permissions 23

Chapter 4. Scoring and Decision-Support Model 25

4.1 Purchase Readiness and Overall Evaluation Model 25

4.2 Priority Score and Base Score 26

4.4 Έκπτωση (Discount) 27

4.5 Ευελιξία Προϋπολογισμού (Budget Flexibility) 28

4.6 Budget Fit and Score Adjustment 29

4.7 Product Categories and Grouping 30

4.8 Value for Money (VFM) 32

4.9 Seasonality: Filtering and Participation in Evaluation 33
4.10 Availability 34
4.11 Implementation of Result Explainability 34
Chapter 5. Data Collection, Cleaning, and Persistence 37
5.1 Product Card Detection and Validation 38
5.2 Title, Link, and Image Extraction 39
5.3 Price Extraction and Normalization 40
5.4 Availability Detection 40
5.5 Discount Detection 41
5.6 Identification, Merging, and State Persistence 41
Chapter 6. Purchase Priorities and Complementary Recommendations προτάσεις 44
6.1 From Individual Products to Purchase Priorities 44
6.2 Purchase Priority Types and Their Purpose 44
6.3 Budget Constraints and Category Diversity 45
6.4 Rejection of Outdated States 46
6.5 Best Next Purchase and Suggestions 47
6.6 Complementary Recommendations 47
Chapter 7. User Interface and Interaction Flow 49
7.1 Interaction Flow 49
7.2 Presentation of Product Evaluation 50
7.3 Presentation of Purchase Priorities 51
7.4 Presentation of Special Cases in the Interface 52
7.5 Complementary Recommendations 52
Chapter 8. Evaluation 54
8.1 Evaluation Objectives and Methodology 54
8.2 Handling Edge Cases 54
8.3 Automated Tests 55
8.4 Regression Tests and Quality Checks 56
8.5 Quantitative Metrics of Objective Achievement 57

8.6 Relationship Between Results and Objectives 57
8.7 Evaluation Limitations 58
Chapter 9. Conclusions and Future Work 59
9.1 Summary of Conclusions 59
9.2 Value of the Deterministic Approach 59
9.3 Implementation Limitations 60
9.4 Future Development Priorities 60
References 61
Appendix A. Map of Main Code Functions 62
Appendix B. Summary Table of Automated Tests 64

Chapter 1. Introduction

1.1 Motivation and Problem Definition

The relevant literature does not support the claim that every large set of options necessarily leads to poor decisions. The meta-analysis by Chernev, Böckenholt, and Goodman shows that decision-making difficulty when numerous options are available is influenced by factors such as the complexity of the choice set, the difficulty of the decision task, and uncertainty regarding preferences [1]. In this thesis, this finding serves as motivation rather than a universal assumption: a large wish list can become difficult to manage when it combines heterogeneous products, different needs, and a limited budget. The application specifically addresses this situation.

1.2 Objective of the Thesis

The main objective is to develop a purchase decision-support application that runs in the browser and uses data from an actual favorites list.

1.3 Main Contributions

The thesis primarily contributes to the level of a complete engineering implementation. It does not propose a new general-purpose mathematical model, but combines established ideas of multi-criteria scoring and constrained optimization in a specific environment for real users and connects them to actual input data. The contribution lies in the coherent workflow from data import to the final explanation of the result, as well as in addressing practical challenges, such as identifying product cards within the page's DOM, handling prices, and distinguishing urgent demand from budget constraints. DOM (Document Object Model) refers to the hierarchical representation of a web page's structure, while a product card is considered the distinct section of the page that contains basic product information, such as its title, image, link, and price.

The main contributions of the thesis are organized into the following categories:

Design of a decision-support mechanism. The thesis develops a product evaluation framework that combines the user's personal needs, discounts, budget compatibility, relative value for money, and seasonality. The individual metrics are used to calculate the final score and purchase readiness.

Collection and management of real-world data. The application extracts products from the Skroutz favorites list and addresses practical issues, such as overlapping product categories, alternative prices, and different availability statuses, as well as preserving the user's state during data reimport.

Generation and presentation of purchase priorities. The application generates purchase priorities based on the active budget and different evaluation objectives. At the same time, it applies diversification across product categories and provides explanations for the results and selected priorities.

Evaluation and reliability of the implementation. The implementation is evaluated through 45 automated tests, covering basic functionality, edge cases, and regression tests. In this way, the application's consistency with the designed rules and constraints is verified.

1.4 Scope and Assumptions

The final version is deliberately limited to Skroutz. The initial design exploration considered broader support for multiple online stores, external interfaces, more complex economic modeling, and product lifecycle mechanisms. These ideas were not incorporated into the final implementation and are not presented as implemented features. Selecting a specific online store allows for more precise DOM inspection and more reproducible development, at the cost of reduced generality.

Furthermore, the application assumes that the urgency score assigned by the user reflects their own judgment. The system does not attempt to prove that this judgment is

economically or behaviorally correct. Similarly, the value-for-money indicator is relative to prices and other products within the same detected category and does not constitute a comprehensive evaluation of quality, technical specifications, or prices across the entire market. These assumptions limit the interpretation of the results and are explicitly stated to avoid unsupported claims.

1.5 Structure of the Thesis

Chapter 2 presents the theoretical background, Chapter 3 covers requirements analysis and system design, while Chapters 5–7 present the main aspects of the implementation and interfaces. Chapter 3 describes the functional and non-functional requirements, as well as the fundamental design choices and initial system architecture. Chapter 4 presents the scoring model and decision-support system. Chapter 5 describes data collection, cleaning, and preservation. Chapter 6 analyzes purchase priorities, budget constraints, and supplementary recommendations. Chapter 7 presents the user interface and the interaction flow. Chapter 8 evaluates the system through automated tests and examines the achievement of the thesis objectives. Finally, Chapter 9 summarizes the main conclusions, implementation limitations, and possibilities for future extensions.

Chapter 2. Theoretical Background and Related Work

The general problem examined in this thesis is supporting users in selecting products from a set of alternatives when multiple criteria, personal preferences, and budget constraints must be considered simultaneously. Various approaches have been developed to address such problems, including multi-criteria evaluation and scoring, recommender systems, and constrained optimization methods. Furthermore, since the application retrieves real-world data from e-commerce websites, an appropriate technological background is also required for accessing and processing the content of these pages. For this reason, the chapter presents the fundamental concepts associated with the aforementioned approaches, as well as Chrome extension technologies and DOM access. Subsequently, related applications are examined, and the position of the proposed solution in relation to them is defined.

2.1 Decision Support in Large Sets of Alternatives

When users are required to choose among many alternatives, the difficulty arises not only from the number of available options but also from the fact that these options often need to be compared based on more than one characteristic. In online shopping, for example, a choice may depend simultaneously on personal need, price, discount, availability, and the available budget. Therefore, the problem is not simply one of ranking products, but of systematically considering different criteria that may have varying degrees of importance to the user.

One fundamental category of methods addressing such problems is **multi-criteria decision making (Multi-Criteria Decision Making, MCDM)**. In such problems, alternatives are evaluated simultaneously against more than one criterion. The criteria may have different levels of importance and different units of measurement; therefore, a mechanism is required to enable their joint consideration. The literature on multi-criteria recommender systems examines the use of multiple evaluation dimensions instead of a single overall rating [11], [12].

One of the simplest methods for combining multiple criteria is the **weighted sum model (Weighted Sum Model, WSM)**. Each criterion is assigned a weight representing its relative importance, and the overall evaluation is obtained by combining the individual values. When weights are normalized so that their sum equals one, the same logic can also be expressed as a weighted linear combination or weighted average. The selection of weights is critical because different values can change the ranking order of the alternatives [2].

This approach is relevant to the present thesis because it allows different user preferences to be combined without requiring them to be derived from previous behavior. However, ranking individual products alone is not sufficient when a set of products must be selected under a limited budget. For this reason, multi-criteria evaluation constitutes only one part of the overall problem, while the remaining related approaches are examined in the following sections.

2.2 Recommender Systems, Algorithmic Recommendations, and Transparency

Multi-criteria evaluation provides a way to compare alternatives, but a decision-support system must also organize and present the results in a manner that users can utilize. This is where the problem connects to recommender systems and, more generally, algorithmic recommendations.

Recommender systems organize, filter, or rank available options to present users with items estimated to be relevant to their needs or preferences. Traditional approaches use, among other techniques, collaborative filtering, content-based features, previous interactions, or combinations of different techniques. In multi-criteria recommender systems, the evaluation of an item may be based on multiple individual dimensions rather than a single score [11], [12].

The concept of algorithmic recommendations is broader. Such a system may produce information, rankings, or suggestions that a person considers when making a decision, without autonomously executing the final choice. Therefore, a decision-support system

does not necessarily need to predict user behavior or discover new items. It may instead organize existing options and clearly present the consequences of different criteria.

In this context, **explainability** and transparency of the results are critical issues. A ranking that displays only a final number makes it difficult for users to understand why one option was placed higher than another. Pu and Chen studied explanation interfaces in e-commerce recommender systems and demonstrated that the way the reasons behind a recommendation are presented can influence users' perceptions of the system's competence and their trust in it [4]. Subsequent research on the evaluation of algorithmic recommendations also examined the relationship between process transparency, trust, and recommendation acceptance [5].

The present thesis is more closely aligned with an explainable decision-support system than with a traditional recommender system that predicts preferences or discovers new products. It begins with products that users have already selected and uses explicit rules to evaluate and explain their ranking. However, ranking alone remains insufficient when the final decision involves a combination of multiple products under a limited budget. This problem is examined in the following section.

2.3 Optimization Under Budget Constraints

Evaluating or ranking individual products answers the question of which option appears more important or suitable according to specific criteria. However, when users have a limited budget and wish to consider multiple products, the problem changes: a combination of items must be selected that satisfies the financial constraint and provides the greatest possible overall utility.

A fundamental category of approaches to such problems is **knapsack problems**. In the classical knapsack problem, each item is characterized by a cost and a utility value, and the objective is to select a combination of items with the greatest total utility without exceeding a predetermined capacity or cost limit.

A related variation is the **multiple-choice knapsack problem**, in which the available items are organized into groups and additional constraints are imposed on the selection of items from each group [3]. This approach is relevant to cases where, in addition to the total cost, the composition of the final set of options is also important.

The connection to these problems demonstrates why simply selecting the products with the highest individual scores does not necessarily lead to the most suitable overall combination. A product may have a high score but consume a large portion of the available budget, thereby excluding several other options that, together, offer greater overall utility.

The three approaches presented in Sections 2.1–2.3 therefore address different aspects of the same problem. Multi-criteria evaluation allows products to be compared based on multiple criteria, the logic of decision-support systems and explainability concerns how results are presented and justified, while optimization under budget constraints concerns the selection of product combinations. The proposed application combines elements of these approaches without claiming to be identical to any of them. The specific scoring framework is presented in Chapter 4, and the process of selecting products under budget constraints is described in Chapter 6.

2.4 Chrome Extensions and DOM Access

A **Chrome Extension** is software that extends or modifies browser functionality through a set of HTML, CSS, and JavaScript files and a manifest file. The manifest describes the extension's basic characteristics, such as its initial files, required permissions, and the websites on which specific code is allowed to execute. The current **Manifest V3** specification defines the corresponding execution and permissions model [6], [8].

Of particular importance in extensions are **content scripts**, namely JavaScript files that are injected into websites and can access the content of their documents. The content script is not simply an additional part of the website. It executes in an environment that is isolated from the page's own JavaScript code, but it has access to the **Document**

Object Model (DOM) and can read or modify document elements in accordance with the extension's permissions [6].

The Document Object Model represents an HTML document as a hierarchical structure of objects and nodes. Elements such as links, images, titles, prices, and structural sections of a page can be identified and accessed programmatically through this structure [9]. In the present thesis, this capability is used to identify product cards on the open Skroutz favorites page and extract the information required by the application.

The extension therefore serves as the programming interface between the existing website and the decision-support mechanism. **wishlistScraper.js** is responsible for accessing the DOM and identifying and extracting products, while **decisionLogic.js** implements the application's core logic and manages the interfaces. The state is maintained through chrome.storage.local, namely the storage space provided by Chrome specifically for the extension [7]. The architecture thus enables the application to use data from the open page without requiring a separate application server.

2.5 Positioning of the Proposed Solution

The functionality of the proposed application is related to three categories of existing tools: product wish lists, price-tracking mechanisms, and e-commerce recommender systems.

Product lists, such as Amazon wish lists, allow users to save items they wish to consider or purchase later. Their primary function is to provide an organized and accessible list of personal product selections, rather than necessarily selecting a purchase plan prioritized under a specific budget [15].

Another related category is price-tracking tools. Google's Price Tracking feature, for example, allows users to monitor the price of a product, optionally set a target price, and receive a notification when the price decreases below that threshold [14]. This supports a specific dimension of the purchasing decision, namely the evolution of prices over time, without being equivalent to the multi-criteria ranking of an entire personal list.

Skroutz itself uses recommendation systems to rank and present products, offers, and related content. According to its official description, recommendations may take into account factors such as search activity, previous interactions, favorites, price, discounts, popularity, seasonality, availability, and total cost [13]. This functionality shares common elements with the present thesis in terms of supporting purchasing choices and using multiple factors, but it serves a different purpose. Skroutz organizes and recommends content within its broader platform, whereas the proposed application starts from the user's already established personal list.

The proposed solution is therefore positioned between a simple wish list and a fully personalized recommendation system. Compared to a simple list, it adds **prioritization**, budgeting, explainable scoring, and the generation of product combinations. Compared to a traditional recommender system, it does not attempt to discover new products through a model of multiple users' behavior and does not use machine learning. Preferences are explicitly specified, rules are deterministic, and the factors influencing the result are presented to the user.

Consequently, the main distinguishing feature of this thesis does not lie in a new general-purpose recommendation algorithm, but in the combination of existing concepts within a specific purchase decision-support workflow: importing a real product list, defining personal priorities, applying multi-criteria scoring, checking budget constraints, explaining the ranking, and generating feasible purchase priorities.

Chapter 3. Requirements Analysis and Design

3.1 Functional Requirements

The functional requirements describe the core capabilities that the system must provide during the purchase decision-support process. Their purpose is to capture what the application is expected to perform, from product import to evaluation and presentation of the results, without determining the way these functions are implemented internally.

The first basic requirement concerns the import and update of products from the Skroutz favorites list (**FR-1**). At the same time, the system must manage product data, the user's personal preferences, and the parameters used during the evaluation process (**FR-2**), so that the user's choices are preserved and taken into account in the decision-support process.

The evaluation is based on criteria such as personal need or urgency, discount, compatibility with the available budget, relative value for money, availability, and seasonality. These criteria constitute parameters of the evaluation mechanism and are not treated as separate functional requirements. Based on them, the system must evaluate and rank the products and provide an explanation of the generated results (**FR-3**).

In addition, the application must support the organization of products into personal lists and the preservation of their state (**FR-4**), as well as the generation of purchase priorities under a budget constraint (**FR-5**). Finally, it must present supplementary recommendations and decision-support results (**FR-6**), which operate in a complementary manner to the main product evaluation.

The functional requirements are summarized in Table 3.1.

Code	**Requirement**	**Code**	**Requirement**
FR-1	Import and update of products from the Skroutz favorites list	FR-4	Organization of products into personal lists and preservation of their state
FR-2	Management of products, personal preferences, and evaluation parameters	FR-5	Generation of purchase priorities under a budget constraint
FR-3	Evaluation, ranking, and explanation of products based on defined criteria	FR-6	Presentation of supplementary recommendations and decision-support results

Table 3.1: Functional requirements of the system



Figure 3.1: Schematic representation of the system's main functional flow

3.2 Non Functional Requirements

The non-functional requirements primarily concern predictability, privacy, robustness against incomplete data, and the clear presentation of results. For the same inputs and evaluation parameters, the scoring, purchase readiness, and purchase priority generation calculations must produce the same result, so that the behavior of the decision-support mechanism is consistent and reproducible.

The application must also comply with the financial constraints defined by the user, so that the generated purchase priorities do not exceed the active budget. Furthermore,

data processing and the preservation of the user's state must be performed locally, without requiring data to be sent to an external application server.

The system must remain functional even when certain secondary product attributes are missing or cannot be determined with certainty. In particular, it must distinguish between a product's actual unavailability and the mere absence of information, in order to avoid erroneous conclusions during evaluation.

Finally, the interface must present both the result and the main factors that influenced it in a clear and understandable manner. In this way, users can understand the logic behind the evaluation without requiring knowledge of the implementation's internal details.

3.3 From the Initial Concept to the Final Version

The system's design choices arise from the functional requirements of Section 3.1 and determine how the purchase decision-support process is organized. A fundamental design objective is to maintain simple, understandable, and consistent interaction, in which users have direct control over the basic parameters influencing product evaluation. At the same time, the application's organization aims to clearly present the calculation results and the constraints taken into account, so that the evaluation process remains easy for users to understand and control.

Product import was designed around a specific favorites list, allowing the system to process a clearly defined set of options (**FR-1**). In parallel, the design provides for the management of product data, personal user preferences, and the parameters used during evaluation (**FR-2**). These parameters include the available budget, personal need, and the relative importance of evaluation criteria.

Product evaluation and ranking are organized around a limited number of criteria to avoid excessive interface complexity. The result is not presented solely as a final numerical score but is accompanied by information that allows users to understand the main factors that influenced it. The same design requirement also includes the separate

handling of cases such as product unavailability or incompatibility with the available budget (**FR-3**).

Regarding the organization of options, the design provides for the creation of personal lists and the preservation of product states across successive interactions (**FR-4**). In this way, users' personal selections remain available as product data are updated.

The generation of purchase priorities is also a fundamental design choice of the application (**FR-5**). A purchase priority is defined as a set of products selected according to a specific evaluation objective and subject to the available budget constraint. The design provides for different types of priorities, allowing users to examine alternatives with different emphases, such as urgency, budget compatibility, or discount.

Finally, the presentation of supplementary recommendations and additional decision-support results is provided for (**FR-6**). These recommendations function as supplementary information alongside the main evaluation process and do not replace the user's final choice.

The main design choices and their relationship to the functional requirements are summarized in Table 3.2.

Design Dimension	Design Choice	Related Requirement
Product source	Use of a specific favorites list	FR-1
Data and preference management	Preservation of products, personal preferences, and evaluation parameters	FR-2
Evaluation and presentation of results	Multi-criteria evaluation, ranking, and explanation of results	FR-3
Product organization	Personal lists and state preservation	FR-4
Purchase priorities	Selection of products under a total cost constraint	FR-5
Supplementary options	Presentation of additional recommendations and results	FR-6

Table 3.2: Main design choices and their relationship to the functional requirements

3.4 System Architecture

The application's architecture is organized into distinct layers to separate data collection, processing, and decision-support logic, local state preservation, and the presentation of results to users. This separation limits direct dependencies between the external website and the remaining system functions.

The Skroutz website constitutes the external data source. The **data collection layer** is responsible for identifying products, extracting the required elements, and validating the data before they are used by subsequent layers of the system.

Product data are then transferred to the **processing and decision-support logic layer**. At this layer, products are evaluated and ranked, budget constraints are managed, and purchase priorities are generated. At the same time, the system uses **local storage** to preserve product data, user preferences, and application state.

The **presentation and user interface layer** allows users to configure parameters and displays products, evaluation results, purchase priorities, and their corresponding explanations. To connect the above architectural description with the actual implementation, Figure 3.2 presents the application's functional block diagram. The overall flow is organized into six basic stages: (1) data import and initial extraction, (2) data processing and normalization, (3) merging and state preservation, (4) scoring and evaluation, (5) purchase priority generation, and (6) explanation and presentation of results. Each block identifies the main implementation capabilities required for the corresponding function, while the arrows illustrate the flow of data and parameters between the individual stages. The diagram provides a concise representation of the functional architecture and is not intended to present all internal function calls or auxiliary functions of the code.

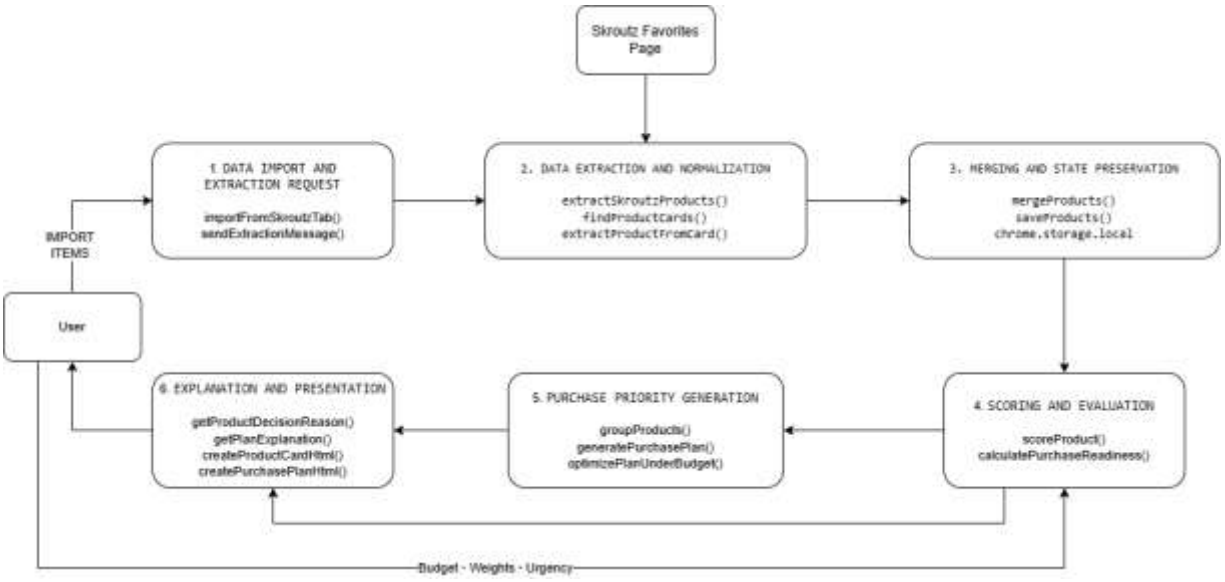


Figure 3.2: Functional block diagram of the application

The functional flow begins with the IMPORT ITEMS action, which initiates the process of importing data from the open Skroutz favorites page. Initially, product cards are extracted and their data are normalized so that the information can be represented in a uniform format before further processing. Subsequently, the imported products are merged with the previously stored state, and the data are preserved locally through chrome.storage.local. During the scoring and evaluation stage, the parameters defined by the user, such as budget, preference weights, and urgency level, are applied to evaluate the products and calculate purchase readiness. The results are then used to generate purchase priorities and the corresponding purchase plan. Finally, the explanation and presentation layer generates the necessary explanations and formats the interface elements through which product evaluations, purchase priorities, and the proposed purchase plan are presented to the user.

3.5 Product Data Model

The data model separates the information originating from the Skroutz website from the state defined by the user for each product. Three basic entities are distinguished: Product, UserProductState, and List. These entities constitute a conceptual description of

the data model. In the JavaScript implementation, they are mainly represented through simple objects and application state fields rather than separate classes.

The Product entity includes the data describing the product: id, title, price, image, url, rawText, availability, and importedAt. Most of these data originate from the DOM of the Skroutz page and can be updated when the product is reimported.

The UserProductState entity describes the user's personal state for each product. It includes urgencyScore, isTracked, for participation in purchase priorities, and listName, which links the product to a personalized list. These fields are preserved during reimport so that updating the data does not affect the user's personal choices.

The List entity represents a personal product list. A list may contain multiple products, while a product may optionally be associated with a personalized list through the listName field. The application also maintains a list registry in local storage.

Entity	Fields	Description
Product	id, title, price, image, url, rawText, availability, importedAt	Basic product data and attributes
UserProductState	urgencyScore, isTracked, listName	Personal choices and user state for the product
List	List name	Grouping of products into a personal list

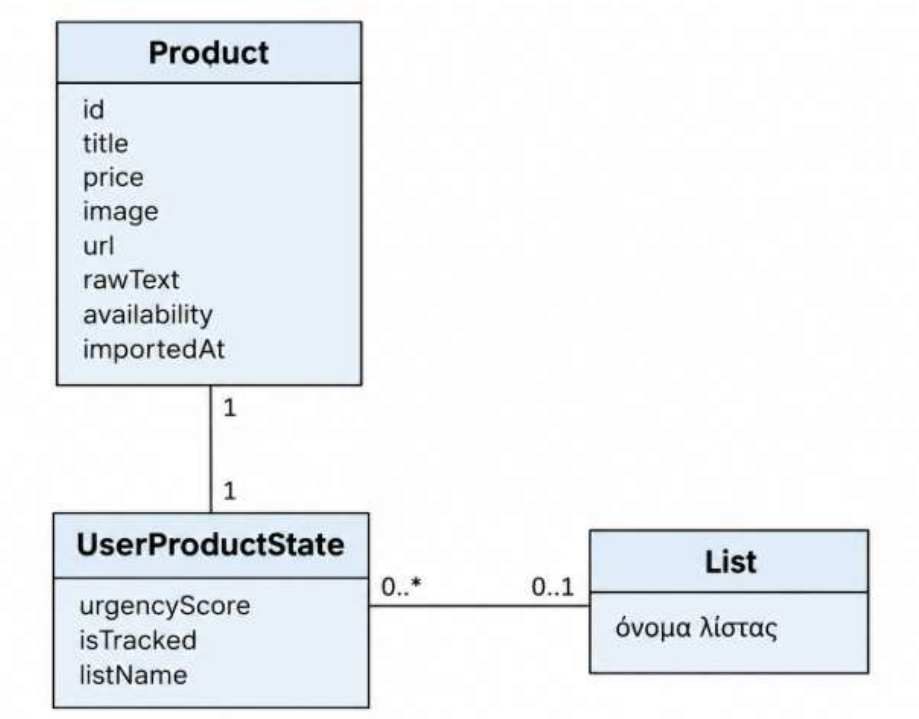
Table 3.3: Basic entities of the data model

The relationship between the entities is as follows: each product has a corresponding user state, which stores the personal choices and settings associated with that specific product, while a personal list may contain multiple products. The association of a product with a personalized list is implemented through the listName field, which determines the list to which the user has assigned the product. In this way, the data originating from the external website are clearly separated from the data expressing the user's personal state and choices.

This distinction is important for the correct preservation of application state, as it allows the external attributes of a product, such as price, availability, or other information

originating from the Skroutz page, to be updated without affecting the settings already defined by the user. Thus, when a product is reimported or updated, the data that have changed are refreshed, while the urgency score, tracking status, and personal list to which the product belongs are preserved. In this way, the loss of the user's personal choices is avoided, and greater consistency in application state management is ensured.

The schematic representation of the entities and their relationships is presented in Figure 3.3.



Σχήμα 3.3: Οντότητες και συσχετίσεις του μοντέλου δεδομένων

3.6 Privacy and Permissions

The application's main security and privacy requirements are limiting data collection and transmission, processing user information locally, minimizing extension access permissions, and securely presenting data originating from external websites. In particular, the application must not require an external application server, an account system, or data storage in a cloud service.

At the design level, product data, personal preferences, and user-defined parameters are stored and processed locally within the browser. In this way, the need to transfer data to external systems is limited. However, choosing local storage does not inherently provide encryption or complete protection of the data stored on the user's device.

The extension's access to external websites is designed according to the principle of least privilege. Access is restricted to the pages necessary for collecting product data, thereby avoiding the granting of broader permissions than those required for the system's functionality.

Finally, data originating from an external website must be presented in a manner that prevents the interpretation of untrusted content as part of the application's commands. The specific mechanisms through which local storage, permission restrictions, and secure data rendering are implemented are presented in the implementation chapter.

Chapter 4. Scoring and Decision-Support Model

4.1 Purchase Readiness and Overall Evaluation Model

Purchase Readiness is a percentage indicator that summarizes the 1–5 scoring scale. It is calculated from five components: 40% final priority score (**Priority Score**), 25% budget compatibility (**Budget Fit**), 15% value for money (**VFM**), 10% seasonality (**Seasonality**), and 10% availability (**Availability**). Priority, Budget Fit, and VFM scores are linearly converted into percentages. Seasonality takes a value of 80% when no specific seasonal filter is applied, 100% when the product matches the selected filter, and 30% when it does not.

For a product with an availability status of available, the Availability component takes a value of 100% and participates in the standard weighted calculation. If the availability status is out-of-stock or unknown, or if no valid positive price exists, Purchase Readiness becomes 0% before the weighted formula is applied. Specifically, for an out-of-stock status, the **Out of Stock** label is displayed, while for an unknown status or the absence of a valid price, **Price Unavailable** is displayed. If the product is available and has a valid positive price, but its price exceeds the active budget, Purchase Readiness becomes 10%, and the **Too Expensive** label is displayed.

For cases that pass the above checks, Purchase Readiness is calculated using the following equation:

Readiness = 0,40·Priority% + 0,25·BudgetFit% + 0,15·VFM% + 0,10·Season% + 0,10·Availability%

The textual categories are **Strong Buy** for a final readiness score of 80% or higher, **Good Option** from 65% to 79%, **Maybe Later** from 45% to 64%, and **Low Priority** below 45%, while the special statuses **Too Expensive**, **Out of Stock**, and **Price Unavailable** operate separately from these categories. These labels serve as distinguishing indicators rather than financial recommendations. The application provides decision-support tools, and the user retains responsibility for the final decision.

Component	Basic Value	Contribution to Readiness
Priority Score	1–5	40%
Budget Fit	1–5	25%
VFM	1–5	15%
Seasonality	80%, 100% ή 30%	10%
Availability	available: 100% · out-of-stock/unknown: Readiness 0%	10%

Table 4.1: Overall framework of criteria and scoring

4.2 Priority Score and Base Score

The base preference score is calculated from the urgency level and discount level, taking into account the corresponding weights defined by the user. If w_u is the weight for urgency and w_d is the weight for discount, the base score is calculated as follows:

BaseScore = (Urgency × w_u + Discount × w_d) / (w_u + w_d)

when $w_u+w_d>0$. When both weights are zero, the neutral value of 3 is used.

Only criteria with positive weights contribute to the calculation. A weight of 0 means that the corresponding criterion does not participate in the calculation of the weighted score.

The use of a weighted combination is related to the classical logic of multi-criteria ranking, in which the relative importance of each criterion is expressed through weights [2]. In the present application, the formula remains simple so that the effect of a change is visible. We do not attempt to learn weights from user behavior.

The base score is then adjusted according to the relationship between the product's price and the active budget. In this way, a product that consumes a small portion of the available amount may be slightly favored, while a product that approaches or exceeds the available limit may receive a negative adjustment. The final priority score (**Priority Score**) is calculated as follows:

PriorityScore = clamp(BaseScore + BudgetAdjustment, 1, 5)

where the clamp function restricts the result to the range 1–5.

The Priority Score is not interpreted as an objective measure of product quality. It expresses only the product's priority according to the user's stated preferences and the application's deterministic rules.

The two direct components of the Base Score, Urgency and Discount, are presented in the following sections, while the budget adjustment is analyzed in Section 4.6.

4.3 Urgency and Preference Weight

Each product starts with a default urgency level (Urgency) of 3/5. The user can modify this value from the product card on a scale of 1–5. This parameter expresses the personal priority that the user assigns to purchasing the specific product and does not constitute an automatic estimate of actual need or purchase probability.

At the same time, two sliders determine the relative importance of Urgency and Discount when calculating the base score. The corresponding weights range from 0 to 5. A higher weight increases the relative influence of the corresponding criterion on the Base Score, while a weight of 0 means that the criterion does not participate in that calculation.

The choice of two directly controllable criteria is deliberate. The initial exploration included more settings, but the final interface maintains a limited, user-friendly set of configuration options. Parameters such as the active budget and VFM arise from the available data and the application's deterministic rules.

4.4 Έκπτωση (Discount)

Discount is the second component of the base weighted score. Unlike Urgency, it is not directly defined by the user but is derived from the product's available information. The application attempts to identify whether a price reduction exists and, when possible, the discount percentage from information such as an explicit discount percentage, a

monetary discount amount, or a combination of an old and a current price. The technical process for detecting these elements is described in detail in Section 5.5.

When a specific discount percentage can be determined, it is converted to a score of 1–5 according to Table 4.2.

Detected Discount	Interpretation
< 10%	1/5
10%–19,99%	2/5
20%–34,99%	3/5
35%–59,99%	4/5
≥ 60%	5/5

Table 4.2: Mapping of discount percentages to Discount scores

If a clear indication of an offer or discount exists but a specific percentage cannot be calculated reliably, the neutral value of 3/5 is used. If no indication of a discount is detected, the value 1/5 is used.

The influence of the Discount Score on the base score is determined by the corresponding preference weight (\(w_d\)), which the user sets on a scale of 0–5. A higher weight increases the relative influence of the discount on the Base Score, while a weight of 0 excludes it from that calculation. This distinction separates a product's detected discount from the importance the user chooses to assign to it.

4.5 Budget Flexibility

The user defines a **base budget (Base Budget)** and selects a **budget flexibility percentage (Budget Flexibility)**, which can take one of the following values {0%, 10%, 30%, 50%, 100%}. The Budget Flexibility parameter expresses the percentage of tolerance the user allows for exceeding the initial budget. Its value does not change the base budget but determines the additional financial margin that the system may take into account when evaluating products.

Based on these two values, the effective budget (Effective Budget) is calculated, representing the maximum financial limit used by the application when checking budget compatibility. The calculation is performed according to the following formula:

$$\mathbf{EffectiveBudget = BaseBudget \times (1 + BudgetFlexibility / 100)}$$

For example, if the base budget is €300 and Budget Flexibility is set to 100%, the effective budget becomes €600. Conversely, when flexibility is 0%, the effective budget is equal to the base budget.

Therefore, the Budget Flexibility parameter acts as a tolerance parameter for the financial constraint, allowing the system to consider products that exceed the initial budget only within the additional financial margin selected by the user.

4.6 Budget Fit and Score Adjustment

The effect of the effective budget is expressed through two distinct mechanisms. Budget Fit constitutes an independent rating on a scale of 1–5 and contributes a weight of 25% to Purchase Readiness. In contrast, Budget Adjustment is not a separate component of Purchase Readiness but is added to the Base Score and modifies the Priority Score. In this way, budget compatibility affects both the overall purchase readiness and the final prioritization, without confusing their respective roles.

The budget compatibility rating calculates how much of the effective budget a product occupies. At the same time, an adjustment is applied to the base score according to the percentage of the effective budget required to purchase the product. Through this distinction, the user sees budget compatibility as an independent indication, while the Priority Score reflects the risk of consuming a large portion of the available amount.

This approach allows the system to maintain the distinct role of the budget without turning it into the sole evaluation criterion. Thus, a product may have high importance for the user but still receive a negative adjustment when its price significantly limits the remaining available options. Conversely, products requiring a small portion of the available amount may be slightly favored, without this implying that they are necessarily

better choices. In this way, the financial dimension is incorporated into the overall evaluation in a controlled and predictable manner.

Effective Budget Usage	Budget Fit
≤ 10%	5/5
≤ 25%	4/5
≤ 40%	3/5
≤ 60%	2/5
> 60%	1/5

Table 4.2: Mapping of effective budget usage to Budget Fit rating

Effective Budget Usage	Final Score Adjustment
≤ 5%	+0,50
≤ 10%	+0,25
≤ 40%	0
≤ 60%	−0,50
≤ 80%	−1,00
≤ 100%	−1,50
> 100%	−2,00

Πίνακας 4.3: Προσαρμογή τελικής βαθμολογίας βάσει χρήσης ενεργού προϋπολογισμού

The final purchase necessity score is calculated as `clamp(BaseScore + BudgetAdjustment, 1, 5)`. The clamp function restricts the result to the limits of the scale. The system does not interpret the final score as an objective measure of product quality. It expresses only purchase necessity according to the specific rules and data of the current list.

4.7 Product Categories and Grouping

The application uses deterministic product categorization so that products with similar roles can be compared and treated as belonging to the same group. Categorization is

performed by the `detectProductGroup()` function, which examines the product title and searches for predefined Greek and English keywords. No machine learning model or semantic analysis is used. If none of the known terms is detected, the product is assigned to the Other category.

The categories as of October 29 in the final application are Audio, Charging Accessory, Case, Phone, Computer, Winter Clothing, Kitchen, Study, Fitness, Home Office, Travel, and Other. The main terms used for matching are presented in Table 4.5.

Category	Main Detection Terms
Audio	airpods, earbud, earbuds, headphone, headphones, ακουστικά
Charging Accessory	φορτιστής, charger, cable, power bank
Case	θήκη, case, cover, σιλικόνης
Phone	iphone, galaxy, smartphone, κινητό
Computer	macbook, laptop, monitor, keyboard, οθόνη
Winter Clothing	jacket, coat, μπουφάν, παλτό
Kitchen	pan, pot, blender, κατσαρόλα, τηγάνι
Study	book, notebook, βιβλίο, τετράδιο
Fitness	gym, fitness, dumbbell, βαράκια
Home Office	desk, chair, γραφείο, καρέκλα
Travel	backpack, suitcase, βαλίτσα, τσάντα
Other	προϊόντα χωρίς αντιστοίχιση στις παραπάνω κατηγορίες

Table 4.5: Product categories and main detection terms

For Latin-script terms, word-boundary checking is applied to reduce incorrect matches when a short term appears as part of a longer, unrelated word. This behavior is particularly important for terms such as pan and book.

Categorization is initially used in the VFM calculation. The price of each product is compared only with the prices of other imported products assigned to the same

category. Thus, for example, the price of a smartphone is not compared with the price of a case or charger. If fewer than two products with valid positive prices are available in the same category, VFM receives the neutral value of 3/5.

The same categorization is also used when generating purchase priorities. The `groupProducts()` function organizes products by category, and the purchase plan generation process treats each category as a separate group. The final purchase priority list may therefore select at most one product from each category. This restriction applies only to the generated purchase plan and not to the products the user can keep in their personal lists. For example, the user may have more than one smartphone saved, but a specific purchase plan can select at most one product from the Phone category.

This approach constitutes a simple rule for differentiating results, without guaranteeing complete or semantically accurate categorization, as it relies exclusively on predefined rules and keywords.

4.8 Value for Money (VFM)

Value for Money (VFM) is a rating on a scale of 1–5 and contributes a weight of 15% to Purchase Readiness. Its calculation is based on the product category previously determined according to Section 4.7.

The Value for Money rating compares the price of a product with the average price of imported products belonging to the same detected category. If fewer than two comparable products are available, the neutral value of 3 is returned. When a sufficient set is available, the ratio price/averageGroupPrice is calculated and mapped to a scale of 1–5.

Price / Average Category Price	VFM
≤ 0,70	5/5
≤ 0,85	4/5
≤ 1,00	3/5

Price / Average Category Price	VFM
≤ 1,15	2/5
> 1,15	1/5

Table 4.6: Mapping of relative product price to VFM rating

The metric is relative rather than absolute. If a list contains two different models with a significant difference in quality, the lower price may result in a higher VFM without implying that the cheaper product is necessarily a better purchase. For this reason, the application does not use VFM as the sole criterion, and this thesis does not present it as a comprehensive economic evaluation.

4.9 Seasonality: Filtering and Participation in Evaluation

Seasonality has two distinct roles in the application: it functions as a product display filter and simultaneously contributes as a component with a weight of 10% to the calculation of Purchase Readiness.

Detection is performed by the `detectProductSeasons()` function, which examines the product title and the available product text (rawText). Explicit lists of Greek and English keywords are used for the seasons Winter, Spring, Summer, and Autumn. A product may be associated with more than one season if multiple rules are triggered. If no seasonal term is detected, the product is classified as All Seasons.

At the interface level, the user can select All Seasons, Winter, Spring, Summer, or Autumn. When All Seasons is selected, no display restriction is applied. If a specific season is selected, `productMatchesSeason()` allows only products for which the corresponding season has been detected to appear in the main view. For example, when the Winter filter is active, only products for which `detectProductSeasons()` has returned the value winter are displayed.

Seasonality does not modify the Priority Score. It contributes only to Purchase Readiness with a weight of 10%. When the All Seasons filter is active, the seasonality component receives a value of 80%. When a specific filter is active, products matching the selected

season receive 100%, while products that do not match receive 30%. The application does not use external weather data and does not attempt to predict seasonal demand.

The seasonal filter concerns product display and does not constitute an independent constraint in the purchase plan optimization process.

4.10 Availability

Availability is a specific parameter of the evaluation mechanism. During data extraction, each product is assigned one of three states: available, out-of-stock, or unknown. The technical process for detecting these states is presented in Section 5.4.

When a product is classified as available, the Availability component receives a value of 100% and contributes a weight of 10% to the normal calculation of Purchase Readiness.

The out-of-stock and unknown states are handled before the normal weighted formula is applied and do not receive the 100% Availability contribution corresponding to the available state. Specifically, if a product is classified as out-of-stock, Purchase Readiness becomes 0% and the Out of Stock indication is displayed. If the availability state is unknown, meaning that availability has not been confirmed or no valid positive price exists, Purchase Readiness also becomes 0% and the Price Unavailable indication is displayed.

Therefore, Availability does not function only as a numerical component with a weight of 10%, but also as a prerequisite for whether a product can proceed through the normal Purchase Readiness calculation process.

4.11 Implementation of Result Explainability

Explainability is implemented as a deterministic presentation layer built on top of the scoring results and is not based on a separate artificial intelligence model. After evaluating a product, the `scoreProduct()` function calculates and returns, along with the product, the individual ratings for Budget Fit, Urgency, Discount, and VFM, as well as the

final score, the budget-based adjustment, and, when available, the detected discount percentage. These data are used directly by the explanation mechanism so that visual and textual indications are derived from the same values used for ranking.

The first form of explanation is implemented by the `generateTags()` function, which maps specific conditions to concise explanatory labels. A final rating of at least 4/5 produces the **Top Priority** indication, a Budget Fit rating of at least 4/5 produces **Budget Friendly**, an Urgency rating of at least 4/5 produces **High Urgency**, and a VFM rating of at least 4/5 produces **Strong VFM**. If the discount rating is at least 4/5, either the actual detected discount percentage or the general **Good Deal** indication is displayed when a specific percentage cannot be reliably calculated. If none of the above rules is triggered, `generateTags()` uses the Lower Priority indication. Cases involving unavailability or the absence of a valid price are handled earlier by `scoreProduct()`, which directly returns specific indications such as Out of Stock or Price Unavailable instead of applying the normal tag generation process.

The second form of explanation is the textual justification generated by the `getProductDecisionReason()` function. The function examines the same individual ratings and dynamically composes a sentence containing the reasons that were triggered for the specific product. These may include a high urgency rating, a strong or **above-average** discount, good budget compatibility, high VFM, and the positive or negative adjustment resulting from the effect of the price on the effective budget. If no specific rule is triggered, a general justification is displayed stating that the selection resulted from the current weighted ratings and budget rules.

The third form is the numerical and visual representation of the rating. The `createProductCardHtml()` function incorporates the final score, purchase readiness, textual justification, explanatory labels, and four horizontal bars for Budget Fit, Urgency, Discount, and VFM into the product card. Each bar is generated by `createBarRow()`. Its value is normalized to the 1–5 scale, and the bar width is calculated as rating / 5 × 100, while the corresponding numerical value is also displayed in the form x/5. When a

specific discount percentage has been detected, it is also displayed in the label of the corresponding bar.

Explainability is also applied at the overall purchase priority level. The `getPlanExplanation()` function generates a different justification depending on the active priority plan. For example, Budget Plan explains that emphasis is placed on budget compatibility, priority, and VFM, while Deal Hunter Plan explains the increased weight assigned to discounts. The explanation displayed by `createPurchasePlanHtml()` in the **Why this plan?** section is therefore based on the active plan, while `getProductDecisionReason()` is reused for each selected product. In this way, the application explains both the evaluation of individual products and the manner in which the overall purchase priority list was generated.

Therefore, the application's explainability is based on rules rather than explainable artificial intelligence. Explanations are not generated retrospectively by an independent mechanism but are produced from the actual ratings and conditions used by the application's deterministic logic. Thus, the user can link the final ranking to the individual values and rules that influenced it.

Chapter 5. Data Collection, Cleaning, and Persistence

Chapter 3 presented the system architecture, the data model, and the basic requirements governing the collection, processing, and persistence of information at the design level. The description in that chapter focuses primarily on how the individual components of the application are organized and the relationships between its main elements, without analyzing in detail how the corresponding functions are implemented in code. This chapter describes the corresponding implementation at the functional and data levels, emphasizing the process through which product information is identified, extracted, validated, normalized, and ultimately incorporated into the application's local state.

The process begins by identifying candidate product cards on the Skroutz favorites page and validating them to reduce the likelihood of importing website elements that do not correspond to actual products. For valid cards, basic product information is then extracted, such as the title, link, image, and price, while additional checks are performed for information concerning discounts and availability. Since these data originate from website content and may appear in different formats, appropriate preprocessing and normalization are required before they can be used by the application's subsequent mechanisms.

This normalization is particularly important for fields such as price and product URL, as these data are subsequently used in budget calculations, product identification, and the prevention of duplicate entries. After the extraction process is completed, the new data are merged with the application's existing stored state. During this process, changing attributes originating from the website are updated, while the user's personal preferences are preserved, such as the urgency rating, a product's inclusion in purchase priorities, and its association with a personal list. In this way, reimporting products can refresh their information without overriding the settings defined by the user.

The main functions involved in this process and their interactions are summarized in Table 5.1.

Stage	Main Function	Role	Next Stage
Product Detection	findProductCards()	Detection of candidate product cards in the DOM	Validity Check
Card Validation	looksLikeProductCard()	Verification that the element corresponds to a product	Data Extraction
Product Extraction	extractProductFromCard()	Creation of the product's basic data	Individual Field Processing
Price Processing	findBestPriceText(), parseGreekPrice ()	Price detection and normalization	Product Identification
Discount Analysis	analyzeDiscount()	Identification and conversion of discounts	Product Evaluation
Availability Check	detectAvailability()	Classification of products as available, out-of-stock, or unknown	Product Evaluation
Import Coordination	extractSkroutzProducts()	Coordination of the overall extraction process	Transfer of Data to the Extension
State Merging	mergeProducts()	Merging new data with the user's stored preferences	Local Storage and Evaluation
Budget Flexibility Management	getEffectiveBudget(), getEffectiveBudgetCents()	Calculation of the effective budget from the base budget and the selected Budget Flexibility	Product Evaluation and Purchase Priority Generation

Table 5.1: Main functions involved in data collection and processing and their interactions

5.1 Product Card Detection and Validation

The import process is coordinated by the `extractSkroutzProducts()` function, which uses `findProductCards()` to identify candidate elements in the main section of the page. Searching the DOM alone is insufficient, as a website may contain promotional sections,

navigation elements, account information, and other elements that may also include images or links in addition to actual products.

For this reason, `looksLikeProductCard()` performs checks and requires a combination of characteristics indicating that an element represents a real product, such as meaningful text, an image, and a suitable link. Only elements that successfully pass this check are passed to `extractProductFromCard()`, which extracts the individual product data.

This validation mechanism also resulted from regression testing. Early versions could import unrelated interface elements as products. The final version includes additional checks that reject such elements, as well as a separate test that rejects an element with characteristics resembling those of an actual product. The design principle is to reject an ambiguous element before introducing it into the decision-support mechanism, since an incorrectly identified product could affect subsequent calculations.

5.2 Title, Link, and Image Extraction

The `extractProductFromCard()` function collects the basic data of the corresponding product. The title is obtained from the card text and cleaned of redundant whitespace. The link is selected using `findBestLink()`, which searches for the most appropriate link corresponding to the product, while the image is obtained from the corresponding card element.

For product identification, `canonicalizeProductUrl()` is used to remove unnecessary tracking parameters and URL fragments. The normalized URL is used as an identity key when merging and reimporting products, preventing different variations of the same URL from creating duplicate entries, while the original product URL may remain stored in the corresponding field.

5.3 Price Extraction and Normalization

Price is a critical field because it is used in budget management, value-for-money calculations, and purchase priority generation. `findBestPriceText()` collects candidate prices from the product card, while `parseGreekPrice()` handles common European number formats, such as "203,15 €", "203.15 €", and "1.203,15 €".

At the same time, `isAuxiliaryPriceCandidate()` is used to reject monetary amounts that are likely to refer to discounts, installments, or other auxiliary information rather than the product's main price. Therefore, the logic is not based on simply selecting any number appearing in the card, since an element may contain more than one monetary amount.

Within the application's logic, monetary comparisons are performed in integer euro cents using the `toCents()` and `fromCents()` methods, limiting rounding errors involving decimal values. Automated tests verify both the exact equality of price and effective budget and cases in which the available limit is exceeded.

5.4 Availability Detection

The `detectAvailability()` function avoids assuming that a missing price automatically indicates product unavailability. Instead, it searches for explicit textual availability indicators and returns one of three states: **available**, **out-of-stock**, or **unknown**.

This distinction prevents incorrectly equating a product whose availability status has not been established with certainty with a product explicitly marked as unavailable. Therefore, the **unknown** state indicates that availability has not been confirmed, while the **out-of-stock** state is associated with an explicit indication of unavailability. The evaluation of these three states during the calculation of **Purchase Readiness** is performed according to the rules described in Section 4.10.

5.5 Discount Detection

A discount may be presented in different ways on a product card. The `analyzeDiscount()` function examines the title and rawText and searches for an explicit discount percentage, a monetary coupon, or a combination of old and current prices.

When an old and a new price are detected, the discount percentage is calculated as follows:

(old price – new price) / old price × 100

The percentage is then converted into a rating on a scale of 1–5. The implementation includes specific processing of monetary amounts and normalized numbers, since simply searching for the character "%" is not sufficient in cases involving monetary coupons, such as "−15 €".

Detected Discount	Discount Rating
< 10%	1/5
10%–19,99%	2/5
20%–34,99%	3/5
35%–59,99%	4/5
≥ 60%	5/5

Table 5.2: Mapping of discount percentage to discount rating

5.6 Identification, Merging, and State Persistence

After extraction is completed, `mergeProducts()` combines the new data with those already stored in the application. This process allows each new import from the Skroutz favorites page to update the existing stored state rather than creating an entirely new set of products.

A product's identity is determined primarily by its normalized URL and, when this is unavailable or unsuitable, by its normalized title. The use of normalized values limits the

likelihood of duplicate entries when the same product appears with minor differences in its URL or data formatting.

During merging, the product's changing external attributes are updated, meaning information originating from the website that may differ between successive imports. In contrast, information expressing the user's personal preferences is preserved, such as urgencyScore, isTracked, and listName. This separation allows the application to handle external product data differently from the personal state configured by the user. Thus, a change in information extracted from the website does not result in the loss of the urgency rating, tracking status, or personal list to which a product has been assigned.

In this way, a new import can update data originating from the website without resetting personal settings to their initial values. This behavior maintains continuity of interaction, as the user does not need to repeat their selections after each update. Furthermore, merging helps maintain a consistent state even when certain external product attributes change.

Local storage is used to persist products, evaluation weights, the base budget, the Budget Flexibility parameter, the list registry, and general user ideas. When the application starts, stored data are retrieved from the interface so that the user's basic settings and personal preferences can be preserved without requiring reimport.

At the implementation level, the main fields of the data model entities, their origins, and their uses are summarized in Table 5.3.

Entity	Field	Source	Use
Product	Product Identification Key	Τίτλος/URL	Stable Product Identification
Product	title	DOM	Display, Normalization, and Seasonality
Product	price	DOM + Normalization	VFM and Purchase Priorities
Product	image	DOM	Visual Product Identification
Product	url	DOM + Normalization	Opening the Product and Avoiding Duplicates
Product	rawText	DOM	Discount and Availability Detection
Product	availability	Extraction Rules	available, out-of-stock, or unknown Status
Product	importedAt	System	Import Timestamp Information
UserProductState	urgencyScore	User	Urgency Rating 1–5
UserProductState	isTracked	User	Participation in Purchase Priorities
UserProductState	listName	User	Association with a Custom List

Table 5.3: Main fields of the data model entities, their origins, and their uses

Chapter 6. Purchase Priorities and Complementary Recommendations προτάσεις

The purchase priority mechanism allows the application to select and organize a set of products proposed to the user based on a specific evaluation objective and the constraints of the available budget. Purchase priorities do not constitute an automatic purchasing decision but are the result of a decision-support mechanism that helps the user examine which products are appropriate for acquisition.

The concept is directly linked to the application's design objectives: product personalization, compliance with the effective budget, differentiation between categories, and explanation of the results. Each purchase priority list is generated by a different function, so that the resulting lists differ in their emphasis on urgency, budget compatibility, or discounts.

6.1 From Individual Products to Purchase Priorities

The user can add products to a custom list (MY SMART PLAN) or select a predefined plan. User participation is the initial step before the final selection performed by the optimization mechanism. Initially, the user declares which products they are interested in and, subsequently, the application evaluates which of these are more suitable for inclusion in the current purchase priority list, based on the active budget and the selected priority type. Thus, a product may remain stored in the user's list without being selected in a specific execution of the purchase prioritization process.

6.2 Purchase Priority Types and Their Purpose

The final application provides four types of purchase priorities. **MY SMART PLAN** places greater emphasis on the overall priority of products, as reflected by the Priority Score. **Budget Plan** places greater emphasis on budget compatibility, with an equal contribution from the product's priority. **Premium Plan** uses the relatively highest price as a substitute for a "premium" choice within the same category, while **Deal Hunter**

Plan gives greater weight to the discount. The name Premium does not mean that the system recognizes product quality or brand prestige; the criterion is the relative price and the Priority Score.

Purchase Priority Type	Product Scoring Function
MY SMART PLAN	Priority
Budget Plan	0,55·BudgetFit + 0,30·Priority + 0,15·VFM
Premium Plan	0,60·RelativePriceRating + 0,40·Priority
Deal Hunter Plan	0,65·Discount + 0,35·Priority

Table 6.1: Purchase priority types and scoring functions

The selection of predefined priority types also affects the weight assigned to urgency and discounts in the calculation of the overall product rating. Specifically, Budget Plan applies weights of 2 and 4 to urgency and discount, respectively; Premium Plan applies weights of 5 and 1; and Deal Hunter Plan applies weights of 3 and 5. After applying the corresponding weight combination, the product ratings are recalculated, and the final selection of purchase priorities is performed using the scoring function of the corresponding type.

6.3 Budget Constraints and Category Diversity

The present section specializes the algorithmic selection process used to generate purchase priorities under budget constraints. The objective is to determine a clear way in which the overall score and differentiation between categories are converted into a specific product selection process.

The purchase priority generation process organizes products into a predefined category order and rejects choices without a valid positive price.

When there is sufficient budget, a multi-step optimization process is applied to the candidate set of products. The algorithm examines alternative combinations and uses an internal cost and total utility comparison to select products. Since each category is

examined only once, the process allows at most one product from each group. This constraint acts as a category diversification rule and permits, for example, the creation of a purchase priority list containing different types of products instead of similar products from the same usage category.

The main selection criterion is utility - threshold. The threshold is 2.5 for all three predefined purchase priority types. If the initial utility does not exceed this limit, the product is not prioritized. However, the final selection also uses the available budget and the total utility of the selected products, in order to avoid the logic of adding lower-value products merely because there is remaining money available. The specific threshold helps the system avoid filling the plan with products that do not provide sufficient benefit.

6.4 Rejection of Outdated States

Within the above algorithmic process, the number of examined states may be large, as each category contains different possible products. To limit this cost, states that cannot lead to a better final selection are rejected.

For the same cost and the same category progress, a state with higher utility is retained. During merging, states are sorted by cost and considered only if they improve the best utility achieved up to that point. If a state costs more without providing higher utility, it is rejected as an inferior choice and does not need to be considered in the subsequent stages of the process.

In this way, the number of alternative states examined is limited without violating the restriction of the available budget. The algorithm is not compared with specific formulations of the multiple-choice knapsack problem [3], but the present application uses its own mechanism for examining candidate states, based on the ability to select at most one product per category. For this reason, the procedure is described as an algorithm specialized for the application's use case rather than as a formal implementation of the classical multiple-choice knapsack problem.

6.5 Best Next Purchase and Suggestions

The best next purchase is selected based on the utility of the active purchase priority type rather than solely on the ranking of the final score. At the same time, the algorithm takes into account the optimal remaining price of the selected products, the percentage of effective budget usage, the more expensive selected product, the best VFM within the specific category, and the availability of the amount. These details, when applicable, are also incorporated into the explanatory purchase advice.

6.6 Complementary Recommendations

The application organizes monitored products into lists. The main list is My Smart Plan, from which the New Wishlist is formed and where the user adds products. In addition, the user can create additional lists of customized options and associate products with them. The lists are neither subcategories of products nor a separate categorization of them, and they should not be confused with product categories such as Fitness. The categorization of products in the application uses only the rules of the corresponding function, while the lists are a distinct storage and organization mechanism controlled by the user.

During the import of a product into a list, the application records the user's action in the relevant fields of the product state. If the user selects tracking, the product is marked as active for participation in purchase priorities. The product's title is retained as imported (isTracked = true), while the listName field stores the list to which it belongs. The selection of the list does not mean that the user must subsequently purchase the product. It expresses the user's intention to keep the product in an active list, based on the same effective budget.

Complementary recommendations can arise from either the main list or a corresponding user list. When the active list is the main list, the button appears as "ADD TO MY SMART PLAN", while for a custom list, the corresponding name is displayed (e.g., "ADD TO COOKING"). When the add action concerns the main wishlist, the system may initially

display alternative products that the user has not yet tracked, so that the user can select products outside their existing list. If the user selects such a product for addition, the corresponding product is retained without a price, stored in a new list, and does not participate in the purchase priority plan until a real price is obtained. At no point does the application perform an actual purchase of the product.

Chapter 7. User Interface and Interaction Flow

7.1 Interaction Flow

The interface organizes user interaction from product import to the presentation of evaluation results and purchase priorities. The user opens the Skroutz favorites page and, through the **IMPORT ITEMS** option, imports the products into the application.

Next, the user defines the base budget, Budget Flexibility, evaluation weights, and urgency ratings for each product. Each change immediately updates the scores, purchase readiness, and individual evaluation components.

The user can also add products to **MY SMART PLAN** or to customized lists. On the right side of the interface, the best next purchase, the current purchase priority list, the total cost, the remaining amount, and brief explanations are presented.

Figure 7.1 presents the overall layout of the interface, with the application settings on the left, the products in the center, and the purchase priority information on the right.

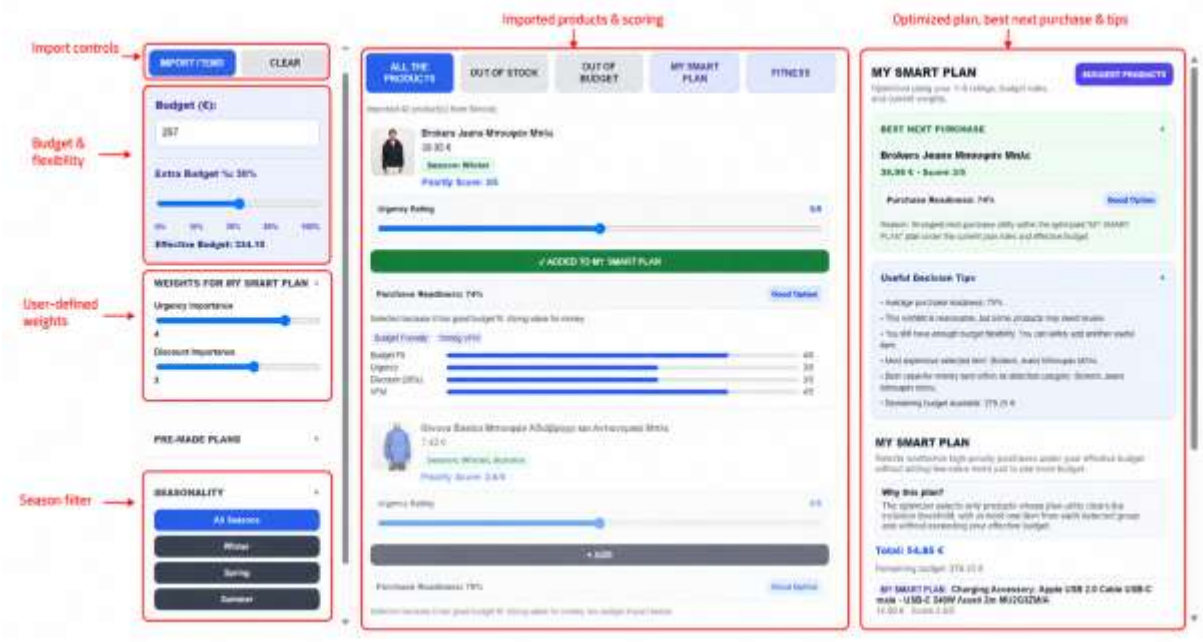


Figure 7.1: General application view showing the budget, weights, product list, and MY SMART PLAN

7.2 Presentation of Product Evaluation

Each product card brings together the essential information required to understand its evaluation. In addition to the title, price, and urgency rating, the final score, purchase readiness, explanatory labels, and individual evaluation components are displayed.

This presentation allows the user to view not only the final result but also the parameters that shaped it. Figure 7.2 presents a representative product card example.

Figure 7.2 focuses on the product card. The final score is not displayed on its own; it is accompanied by purchase readiness, a textual justification, labels, and four components. These elements are generated dynamically during the creation of the product card. The `createProductCardHtml()` function receives the already evaluated product and uses `getProductDecisionReason()` and `createBarRow()` for the textual and visual presentation of the individual results, while the labels have been previously generated by `generateTags()`. Therefore, when the budget, urgency rating, or preference weights change and a new evaluation is performed, the corresponding values and explanations are updated immediately in the interface.

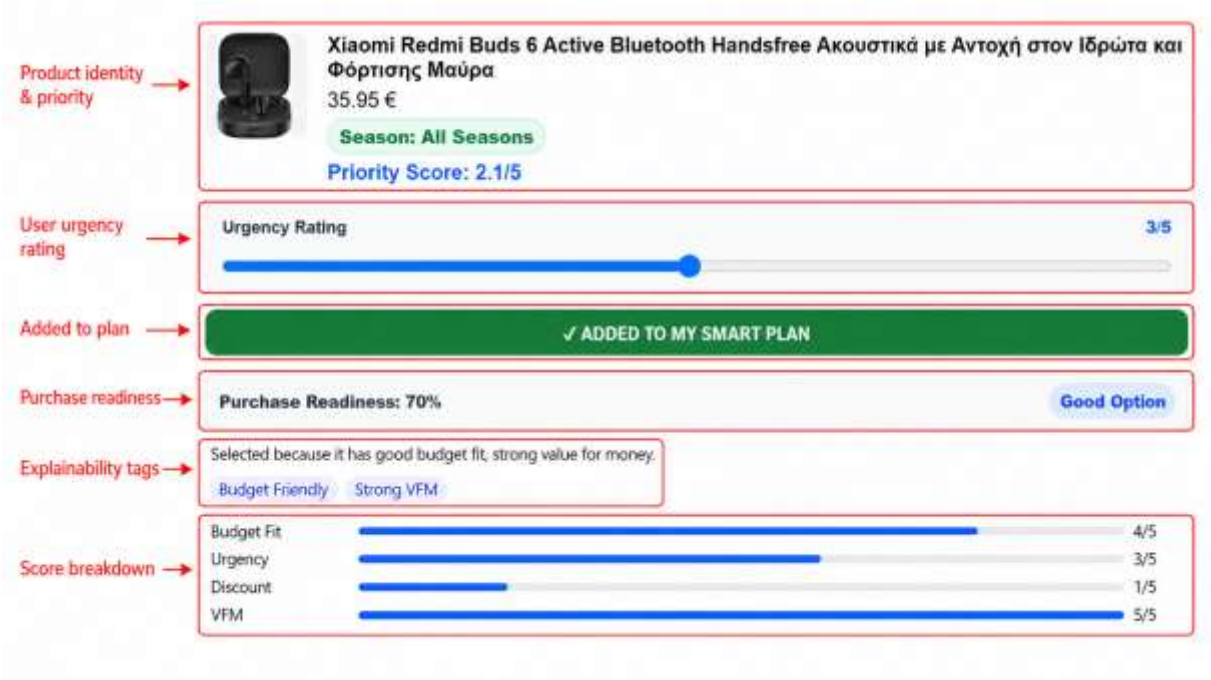


Figure 7.2: Example of product score explanation

7.3 Presentation of Purchase Priorities

The purchase priority section brings together information concerning the results of product selection under the effective budget. The interface displays the best next purchase, the total cost of selected products, the remaining amount, and a brief justification of the specific priority list. The explanation of the overall priority list is generated by `getPlanExplanation()` based on the active purchase priority type and displayed by `createPurchasePlanHtml()` in the Why this plan? section. The rules described vary depending on whether MY SMART PLAN, Budget Plan, Premium Plan, or Deal Hunter Plan has been selected. Furthermore, each selected product is accompanied by its own textual justification through `getProductDecisionReason()`, ensuring that the explanation of the overall plan is combined with the explanation of individual selections.

Figure 7.3 presents an example of MY SMART PLAN and the information accompanying the active purchase priority list.

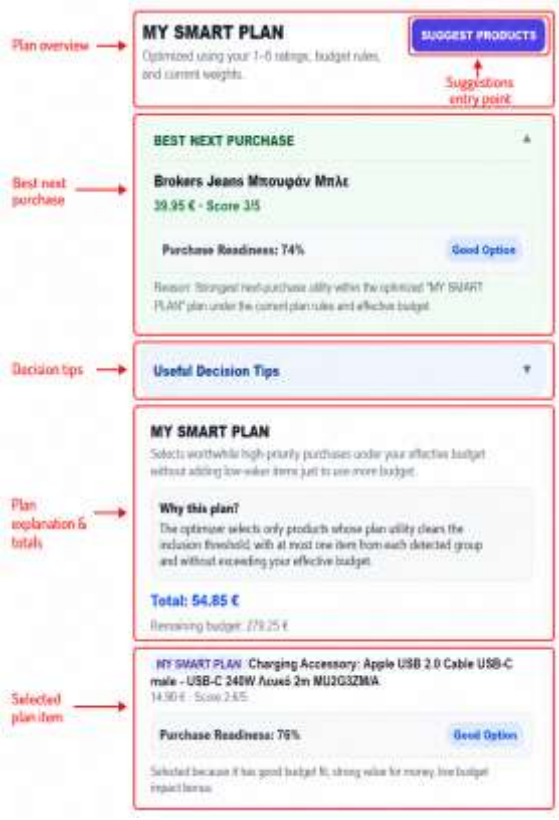


Figure 7.3: Best next purchase and active MY SMART PLAN priority list

7.4 Presentation of Special Cases in the Interface

The interface visually distinguishes cases in which a product has low compatibility with the available budget. When a product's price exceeds the effective budget, a correspondingly low Budget Fit rating, the impact on the score, and the textual indication Too Expensive are displayed.

Figure 7.4 presents a representative example of this case.



Figure 7.4: Analysis of a product with a high impact on the budget

7.5 Complementary Recommendations

The interface presents two distinct categories of complementary recommendations. The first includes existing products from the imported list that can be added directly to the active purchase priority list. The second includes general product ideas that are not treated as actual products with a known price.

The clear distinction between these two categories prevents confusion between actual products in the list and general recommendations. The corresponding interface presentation is shown in Figure 7.5.

Figure 7.5 presents the two categories of recommendations: existing products from the imported list and general ideas. The interface explicitly distinguishes general ideas from actual products with a known price.

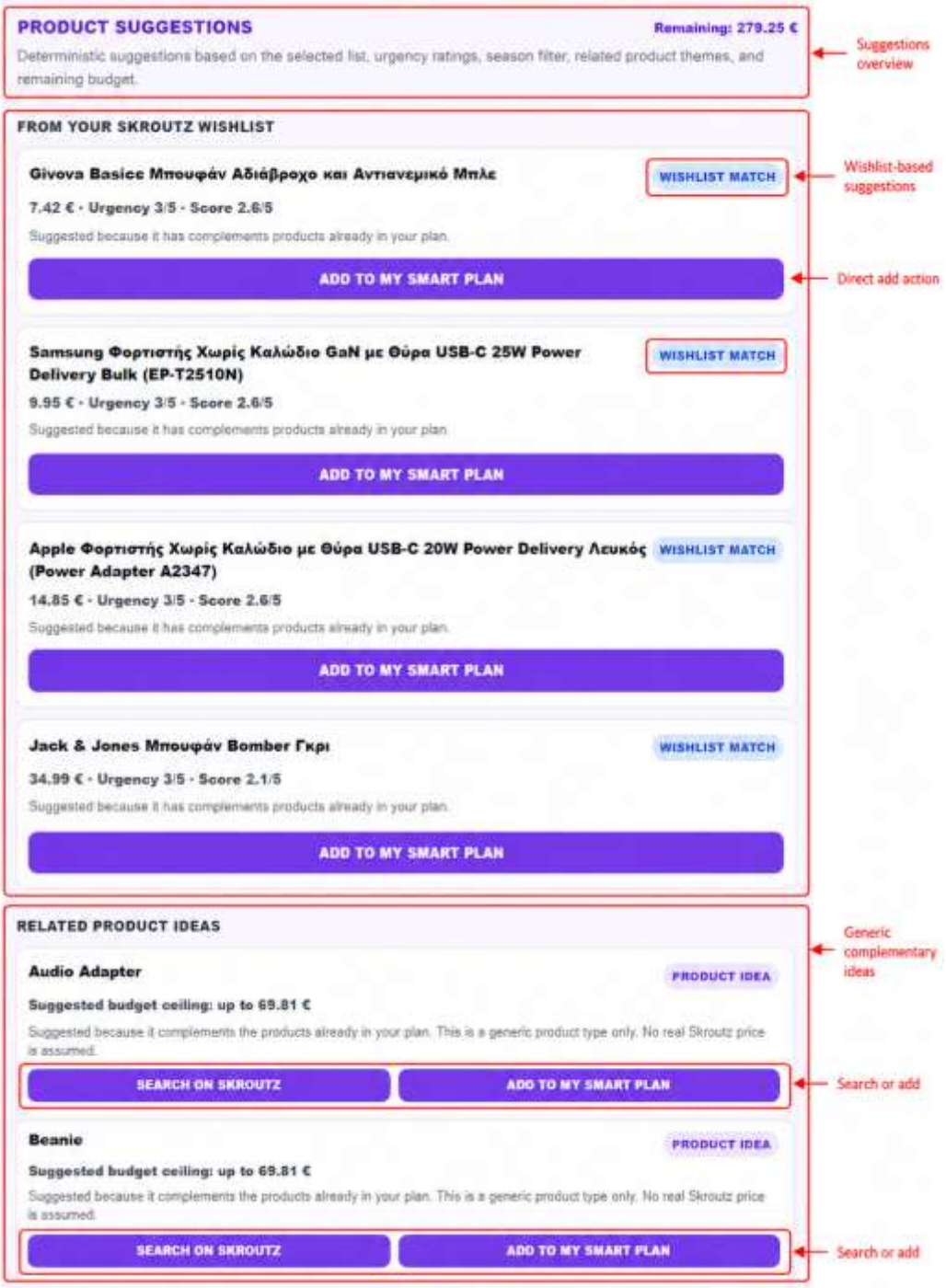


Figure 7.5: Complementary recommendations from the list and general product ideas

Chapter 8. Evaluation

8.1 Evaluation Objectives and Methodology

The evaluation focuses on achieving the technical objectives of the thesis: the correct application of basic rules, the behavior in edge cases, the consistency of budget management, the prevention of false product imports, the preservation of state, and stability against regressions. No external user experiment was conducted and, therefore, metrics such as decision-making time, perceived usability, or changes in user confidence are not used. The evaluation is limited to verifying that the software executes the designed rules consistently in controlled scenarios.

The methodology is based on an automated test suite implemented in Python using the unittest framework. The tests execute the actual JavaScript logic from `src/wishlistScraper.js` and `src/decisionLogic.js` through Node.js, using a lightweight test harness that provides substitutes for the DOM and the basic interfaces of the Chrome Extension API. In this way, individual behaviors, boundary values, regressions, and basic static checks of the repository's structural consistency and security are examined without requiring a complete simulation of the browser environment.

8.2 Handling Edge Cases

System reliability depends more on edge cases than on "normal" scenarios. For this reason, the final version includes explicit behavior for an empty budget, a negative budget, a zero effective budget, a negative income value, exact price-to-limit equality, missing prices, availability, zero weights, URL normalization, false keyword matches, and product reimporting. The existence of separate tests for these cases reduces the likelihood of previously corrected errors reappearing in a subsequent version.

8.3 Automated Tests

The automated suite includes a total of **45 tests**, distributed across five test files. During the final verification, **all 45 tests completed successfully**. This result demonstrates the consistency of the implementation with respect to the specific functions and edge cases covered by the suite, without constituting proof of overall software correctness.

Test File	Number	Main Objects Tested
test_manifest.py	4	Manifest V3, permissions, and host scope
test_content_logic.py	6	Availability, price, and DOM card validation
test_popup_logic.py	15	Discount, weights, budget, VFM, seasonality, optimizer
test_regressions.py	15	Product identity, edge cases, readiness, regressions
test_code_quality.py	5	Isolation of incorrect functions, absence of debugger, absence of embedded hard-coded secrets, and prevention of unnecessary access mechanisms
Total	**45**	**45/45 successful tests**

Table 8.1: Distribution of automated tests by file and main objects tested

Regression tests are particularly important, as they cover behaviors that had caused problems during development. These include incorrect classification of products without a price as out-of-stock, the processing of monetary coupons and old prices for discount calculation, the handling of zero, empty, and negative budgets, the prevention of duplicate product entries caused by variations of the same URL, the avoidance of false matches during categorization, and the prevention of selecting low-utility products solely to exhaust the remaining available budget.

8.4 Regression Tests and Quality Checks

In this thesis, testing is used for the functional verification of specific application behaviors and for checking certain repository properties. A single numerical software quality metric is not used. Automated checks are divided into regression tests and static source code checks.

Regression tests include 15 checks in the test_regressions.py file and verify behaviors that either presented errors during development or are considered critical to application stability. These include URL normalization and identity, avoidance of incorrect matches during categorization, Purchase Readiness behavior at the budget limit, compliance with optimizer constraints, and the exactness of monetary conversions in cents. All 15 tests complete successfully (**15/15 = 100%**).

In parallel, test_code_quality.py performs five static checks: detection of duplicate function declarations in the main JavaScript files (two checks), detection of debugger statements, detection of obvious hard-coded secrets (assignments to names such as api_key, password, secret, and access_token), and verification that no extension domain list contains an entry larger than zero. The expected number of findings is zero in all five cases, and all tests complete successfully (**5/5 = 100%**). The secret detection check does not constitute a complete security analysis but a basic check for obvious cases.

These percentages refer exclusively to the specific scenarios and properties examined by the tests and do not constitute an overall software quality metric.

8.5 Quantitative Metrics of Objective Achievement

Metric	Result	What It Demonstrates	What It Does Not Demonstrate
Automated tests	45/45 = 100%	Consistency of controlled functionality and edge-case checks	Absence of all possible errors
Effective budget violations in optimizer tests	0 in relevant tests	Compliance with the cost constraint in the tested scenarios	Mathematical proof for every possible priority plan
Selections per group in optimizer tests	≤ 1	The category diversification rule was applied in the tested logic	That categorization is semantically perfect
Static repository quality checks	5/5 successful	No duplicate function declarations, debugger statements, obvious hard-coded secrets, or unnecessarily broad host access in the specific checks	Does not constitute a measurement of cohesion, coupling, complexity, or overall code quality

Table 8.2: Quantitative metrics of objective achievement and documentation of results

8.6 Relationship Between Results and Objectives

The **first objective**, reliable data import, is supported by automated tests for product card validation, price extraction and normalization, as well as checks preventing the acceptance of invalid DOM elements. The **second objective**, safe financial logic, is supported by tests covering zero, negative, undefined, and boundary budgets, as well as checks verifying compliance with the effective budget when generating purchase priorities. The **third objective**, adapting rankings to user preferences, is verified through tests of score changes when preference weights change. The **fourth objective**, state persistence, is examined through the automated process of reimporting and merging products, which verifies that properties such as urgencyScore, isTracked, and listName are preserved. Finally, the **fifth objective**, understandable presentation of results, is functionally documented by the visible interface elements and the explanatory

indications accompanying the results. However, the understanding, usability, and perceived trust of users were not evaluated through an experiment involving end users.

8.7 Evaluation Limitations

The main limitation is the absence of a user study. The thesis cannot support claims that users make decisions faster, purchase more "correctly," or trust the system more. The bibliography on explanations provides a relevant motivation [4], [5], but does not replace the evaluation of this specific application. A future study could compare a simple wishlist with the proposed application in terms of decision-making time, perceived difficulty, understanding of the reasons for ranking, and final choice.

A second limitation is that the automated tests do not execute the complete Chrome environment. The harness simulates only the necessary interfaces. A third limitation is the dependence on external DOM structures. A fourth is the use of heuristic rules that have not been validated against a real population of users. Therefore, the final evaluation is strong as a software verification assessment and more limited as an evaluation of human decision-making.

Chapter 9. Conclusions and Future Work

9.1 Summary of Conclusions

This thesis demonstrated that a wishlist can be transformed into a structured decision-support environment without requiring an external advertising service or a machine learning model. The final application imports products from Skroutz, preserves personal settings, calculates deterministic metrics, applies budget constraints, and generates purchase priority lists based on the effective budget. The essential characteristic of the system is not that it knows which product the user should purchase, but that it transforms the user's own preferences and financial parameters into a consistent, transparent, and repeatable decision-support process.

The evaluation results support three main conclusions. **First**, the implementation handles edge cases relating to the budget, availability, and validity of input data consistently in the tested scenarios. **Second**, the scoring and ranking respond to changes in urgency ratings and user preference weights as designed. **Third**, the purchase priority generation mechanism respects the effective budget and the relevant category diversification constraints in the automated scenarios covered by the test suite. The above does not prove that the rules used constitute the only optimal possible approach. However, it demonstrates that the selected deterministic logic has been implemented with internal consistency and predictable behavior in the examined application domain.

9.2 Value of the Deterministic Approach

The deterministic approach proved suitable for the specific objective because it allows a direct connection between input and result. The user can view the urgency rating, the detected discount, Budget Fit, and VFM and understand why a particular option increases or decreases in priority. The literature indicates that transparency and explanations can be important for the acceptance of recommender systems [4], [5]. This thesis adopts this principle in a non-AI system, without claiming that it achieves an equivalent psychological effect.

9.3 Implementation Limitations

Data extraction depends on the structure of Skroutz. If product cards, options, or textual availability indicators change, the scraper may require maintenance. Furthermore, the application does not communicate directly with Skroutz and does not guarantee that every future version of the website will remain compatible. It also does not perform continuous product monitoring in the background and does not automatically update local data without a new import.

Categorization and seasonality are based on explicit keywords. This ensures predictability but limits the coverage of new or ambiguous titles. VFM is based only on relative price within the imported list. Premium Plan uses the higher relative price as a proxy rather than a quality assessment. These limitations must remain clear to the reader, as they differ from weaknesses that could lead to broader conclusions than those supported by the implementation.

9.4 Future Development Priorities

The most important direction for future development is the evaluation of the application with real users, in order to examine whether the ranking explanations, Priority Score, and Purchase Readiness are understandable and whether the application facilitates meaningful purchase decisions.

A second direction is support for additional online stores. The current implementation is adapted to Skroutz; therefore, it could be extended in the future with separate data extraction mechanisms for each website, while maintaining common core scoring and purchasing priority generation logic.

Both extensions are directly linked to the main limitations of the present version and can be evaluated using clearly defined criteria.

References

[1] Chernev, U. Böckenholt and J. Goodman, "Choice overload: A conceptual review and meta-analysis" Journal of Consumer Psychology, vol. 25, no. 2, pp. 333–358, 2015.

[2] R. O'Shea, P. Deeney, E. Triantaphyllou, L. Diaz-Balteiro and S. A. Tarim, "Weight stability intervals for multi-criteria decision analysis using the weighted sum model" Expert Systems with Applications, vol. 296, 128460, 2026.

[3] R. M. Nauss, "The 0–1 knapsack problem with multiple choice constraints" European Journal of Operational Research, vol. 2, no. 2, pp. 125–131, 1978.

[4] P. Pu and L. Chen, "Trust-inspiring explanation interfaces for recommender systems" Knowledge-Based Systems, vol. 20, no. 6, pp. 542–556, 2007.

[5] X. Ning, Y. Lu, W. Li and S. Gupta, "How transparency affects algorithmic advice utilization: The mediating roles of trusting beliefs" Decision Support Systems, vol. 183, 114273, 2024.

[6] Chrome for Developers, "Chrome Extensions – Documentation and Reference" Google.

[7] Chrome for Developers, "chrome.storage API" Google.

[8] Chrome for Developers, "Declare permissions" Google.

[9] MDN Web Docs, "Document Object Model (DOM)" Mozilla. Διαθέσιμο: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model.

[10] C. Gkovaris, "Supporting Online Purchase Decisions," software project, version 1.3.2, unpublished private GitHub repository, 2026.

[11] G. Adomavicius, N. Manouselis and Y. Kwon, "Multi-Criteria Recommender Systems," in Recommender Systems Handbook, Springer, pp. 769–803, 2011.

[12] D. Monti, G. Rizzo and M. Morisio, "A systematic literature review of multicriteria recommender systems," Artificial Intelligence Review, vol. 54, pp. 427–468, 2021.

[13] Skroutz, "Πώς λειτουργούν οι προτάσεις στο Skroutz; (συστήματα συστάσεων)"

[14] Google, "About price tracking on Google" Google Shopping Help.

[15] Amazon, "Create Your List" Amazon Customer Service.

Appendix A. Map of Main Code Functions

This appendix serves as a quick reference between the design chapters and the source code. It does not replace the repository's source code. It records the functions most relevant to the logic of the thesis.

Functionality	File	Main Function(s)
Product Extraction	src/wishlistScraper.js	extractSkroutzProducts(), findProductCards(), extractProductFromCard()
Card Validation	src/wishlistScraper.js	looksLikeProductCard()
Price	src/wishlistScraper.js	findBestPriceText(), parseGreekPrice(), extractEuroPriceCandidates()
Availability	src/wishlistScraper.js	detectAvailability()
Import from Page	src/decisionLogic.js	importFromSkroutzTab(), sendExtractionMessage()
Identification	src/decisionLogic.js	canonicalizeProductUrl(), getProductIdentityKey(), mergeProducts()
Budget	src/decisionLogic.js	sanitizeBudgetInput(), getEffectiveBudgetCents(), getEffectiveBudget()
Scoring	src/decisionLogic.js	scoreProduct(), weightedAverageRating(), calculateBudgetAdjustment()
Discount	src/decisionLogic.js	analyzeDiscount(), discountPercentToRating()
VFM	src/decisionLogic.js	calculateVfmScore()
Seasonality	src/decisionLogic.js	detectProductSeasons(), productMatchesSeason()
Product Groups	src/decisionLogic.js	detectProductGroup(), groupProducts()
Purchase Priorities	src/decisionLogic.js	generatePurchasePlan(), optimizePlanUnderBudget(), pruneDominatedStates()

Functionality	File	Main Function(s)
Purchase Readiness	src/decisionLogic.js	calculatePurchaseReadiness(), getPurchaseReadinessLabel()
Explainability	src/decisionLogic.js	generateTags(), getProductDecisionReason(), getPlanExplanation(), createProductCardHtml(), createPurchasePlanHtml(), createBarRow()
Recommendations	src/decisionLogic.js	getWishlistProductSuggestions(), getGenericProductSuggestions()
Safe Display	src/decisionLogic.js	escapeHtml()

Table A.1: Map of Main Code Functions

Appendix B. Summary Table of Automated Tests

This appendix provides a summary of the automated tests performed to verify the correct operation of the application. The table groups the tests by file, the objects being tested, and the main functions examined, facilitating the connection between the implementation and the system testing and evaluation process.

ID	Test File	Area	Summary
01-04	test_manifest.py	Extension Configuration	Manifest V3, required files, permissions, and restricted access to Skroutz
05-10	test_content_logic.py	Data Collection	Availability, price extraction, product card validation, and rejection of invalid DOM elements
11-25	test_popup_logic.py	Decision-Support Logic	Discount, weights, budget, VFM, seasonality, purchase readiness, and purchase priority generation
26-40	test_regressions.py	Regressions	Product identification and merging, edge cases, readiness, categorization, and optimizer constraints
41-45	test_code_quality.py	Code Quality	Duplicate functions, debugger statements, possible hard-coded secrets, and control of required files

Table B.1: Summary Table of Automated Tests